

Practical 6 — Constraint Satisfaction: Map Colouring and 8-Queens

Run: python P6_csp.py | Library: none, plain Python 3

Program

```
# Practical 6 -- Constraint Satisfaction: Map Colouring and 8-Queens
# Plain backtracking search, no external libraries.

# -----
# Part A: Map Colouring (Australia's mainland states/territories)
# -----

australia_neighbors = {
    "WA": ["NT", "SA"],
    "NT": ["WA", "SA", "Q"],
    "SA": ["WA", "NT", "Q", "NSW", "V"],
    "Q": ["NT", "SA", "NSW"],
    "NSW": ["SA", "Q", "V"],
    "V": ["SA", "NSW"],
    "T": [],
}

def is_consistent(region, colour, assignment, neighbors):
    for other in neighbors[region]:
        if assignment.get(other) == colour:
            return False
    return True

def backtrack_colouring(assignment, regions, colours, neighbors):
    if len(assignment) == len(regions):
        return assignment
    unassigned = [r for r in regions if r not in assignment]
    region = unassigned[0]
    for colour in colours:
        if is_consistent(region, colour, assignment, neighbors):
            assignment[region] = colour
            result = backtrack_colouring(assignment, regions, colours, neighbors)
            if result is not None:
                return result
            del assignment[region]
    return None

def solve_map_colouring():
    regions = list(australia_neighbors.keys())
    colours = ["Red", "Green", "Blue"]
    solution = backtrack_colouring({}, regions, colours, australia_neighbors)
    return solution

# -----
# Part B: 8-Queens
# -----

def is_safe(row, col, queens):
    for r, c in enumerate(queens):
        if c is None:
            continue
        if c == col or abs(c - col) == abs(r - row):
            return False
    return True

def backtrack_queens(row, n, queens, solutions, find_all=False):
    if row == n:
        solutions.append(queens.copy())
        return not find_all # stop after first solution unless find_all
    for col in range(n):
        if is_safe(row, col, queens):
            queens[row] = col
            if backtrack_queens(row + 1, n, queens, solutions, find_all):
                return True
            queens[row] = None
    return False
```

```

def solve_8queens_first():
    queens = [None] * 8
    solutions = []
    backtrack_queens(0, 8, queens, solutions, find_all=False)
    return solutions[0]

def count_all_8queens_solutions():
    queens = [None] * 8
    solutions = []
    backtrack_queens(0, 8, queens, solutions, find_all=True)
    return solutions

def print_board(queens, n=8):
    for r in range(n):
        row = ["Q" if queens[r] == c else "." for c in range(n)]
        print(" ".join(row))
    print()

# -----
# Main
# -----

if __name__ == "__main__":
    print("=" * 55)
    print("PART A: MAP COLOURING (Australia's states/territories)")
    print("=" * 55)
    solution = solve_map_colouring()
    print("Regions and their assigned colours:")
    for region, colour in solution.items():
        print(f" {region:4s} -> {colour}")

    print("\nAdjacency check (no two neighbouring regions share a colour):")
    all_ok = True
    for region, neighbors in australia_neighbors.items():
        for n in neighbors:
            same = solution[region] == solution[n]
            if same:
                all_ok = False
                print(f" {region}-{n}: {solution[region]} vs {solution[n]} "
                      f"-> {'CONFLICT' if same else 'OK'}")
    print(f"\nAll constraints satisfied: {all_ok}")

    print("\n" + "=" * 55)
    print("PART B: 8-QUEENS PROBLEM")
    print("=" * 55)
    first = solve_8queens_first()
    print("First solution found (column index of queen in each row):")
    print(first)
    print("\nBoard:")
    print_board(first)

    all_solutions = count_all_8queens_solutions()
    print(f"Total number of distinct solutions to 8-Queens: {len(all_solutions)}")

```

Output

```
=====
PART A: MAP COLOURING (Australia's states/territories)
=====
```

Regions and their assigned colours:

```
WA  -> Red
NT  -> Green
SA  -> Blue
Q   -> Red
NSW -> Green
V   -> Red
T   -> Red
```

Adjacency check (no two neighbouring regions share a colour):

```
WA-NT: Red vs Green -> OK
WA-SA: Red vs Blue  -> OK
NT-WA: Green vs Red  -> OK
NT-SA: Green vs Blue -> OK
NT-Q:  Green vs Red  -> OK
SA-WA: Blue vs Red   -> OK
SA-NT: Blue vs Green -> OK
SA-Q:  Blue vs Red   -> OK
SA-NSW: Blue vs Green -> OK
SA-V:  Blue vs Red   -> OK
Q-NT:  Red vs Green  -> OK
Q-SA:  Red vs Blue   -> OK
Q-NSW: Red vs Green  -> OK
NSW-SA: Green vs Blue -> OK
NSW-Q: Green vs Red   -> OK
NSW-V: Green vs Red   -> OK
V-SA:  Red vs Blue   -> OK
V-NSW: Red vs Green  -> OK
```

All constraints satisfied: True

```
=====
PART B: 8-QUEENS PROBLEM
=====
```

First solution found (column index of queen in each row):

```
[0, 4, 7, 5, 2, 6, 1, 3]
```

Board:

```
Q . . . . .
. . . . Q . .
. . . . . Q
. . . . Q . .
. . Q . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . .
. . . Q . . .
```

Total number of distinct solutions to 8-Queens: 92